STS CAT-2

Max Sliding Window

```
import java.util.*;

class Main
{
    static public void main(String[] prakash)
    {
        Scanner sc=new Scanner(System.in);
        int n=sc.nextInt();
        int[] arr=new int[n];
        for(int i=0;i<n;i++)
          arr[i]=sc.nextInt();
        int window=sc.nextInt();
        maxSlidingWindow(arr,n,window);
    }
    static void maxSlidingWindow(int[] arr,int n,int window)
    {
        if(arr.length==0 || window<=0)
          return;
        int[] res=new int[n-window+1];
        Deque<Integer> dq=new LinkedList<>();
        for(int i=0;i<n;i++)
        {
            // shrink window from left side
            while(!dq.isEmpty() && dq.peekFirst()<=i-window)
              dq.pollFirst();
            while(!dq.isEmpty() && arr[dq.peekLast()]<arr[i])
              dq.pollLast();
            dq.addLast(i);
            if(i>=window-1)
              res[i-window+1]=arr[dq.peekFirst()];
```

```
        }

        for(int i=0;i<res.length;i++)
          System.out.print(res[i]+" ");
    }
}
```

Trees

Vertical Order Traversal , insert data using bst

TC :

Average
O(n log n)
Worst (skewed BST + TreeMap)
O(n^2)

```
import java.util.*;

public class Main

{
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
     Node root = null;
     int n = sc.nextInt();
     for (int i = 0; i < n; i++)
         root = insert(root, sc.nextInt());
     verticalOrderTraversal(root);
}
static void verticalOrderTraversal(Node root)
{
if(root==null) return;
```

```
Queue<Node> nodeQueue=new LinkedList<>();

Queue<Integer> hdQueue=new LinkedList<>();

nodeQueue.add(root);

hdQueue.add(0);

Map<Integer,List<Integer>> map=new TreeMap<>();

while(!nodeQueue.isEmpty())

{
     Node node=nodeQueue.poll();
     int hd=hdQueue.poll();
     if(!map.containsKey(hd)) map.put(hd,new ArrayList<>());
     map.get(hd).add(node.data);
     if(node.left!=null)
     {
         nodeQueue.add(node.left);
         hdQueue.add(hd-1);
     }

      if(node.right!=null)
     {
         nodeQueue.add(node.right);
         hdQueue.add(hd+1);

     }

}
    for(Map.Entry<Integer,List<Integer>> obj:map.entrySet())
{

    // System.out.print(obj.getKey()+" : ");
     for(int ele:obj.getValue())
```

```
        System.out.print(ele+" ");
      //System.out.println();
}
}
static Node insert(Node root,int data)
{
    if(root==null) return new Node(data);
    if(data<root.data)
       root.left=insert(root.left,data);
    else
      root.right=insert(root.right,data);
     return root;
}
static class Node
{
    int data;
    Node left=null,right=null;
    Node(int d)
    {
        data=d;
    }
}

}
```

Vertical Order Traversal , insert data using build Binary Tree

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        Node root = buildTree(sc.nextLine());
        verticalOrderTraversal(root);
    }

    static void verticalOrderTraversal(Node root) {
        if (root == null) return;

        Queue<Node> nodeQueue = new LinkedList<>();
        Queue<Integer> hdQueue = new LinkedList<>();

        nodeQueue.add(root);
        hdQueue.add(0);

        Map<Integer, List<Integer>> map = new TreeMap<>();

        while (!nodeQueue.isEmpty()) {
            Node node = nodeQueue.poll();
            int hd = hdQueue.poll();

            map.putIfAbsent(hd, new ArrayList<>());
            map.get(hd).add(node.data);

            if (node.left != null) {
                nodeQueue.add(node.left);
                hdQueue.add(hd - 1);
            }

            if (node.right != null) {
                nodeQueue.add(node.right);
                hdQueue.add(hd + 1);
            }
        }

        for (Map.Entry<Integer, List<Integer>> entry : map.en
trySet()) {
```

```
            System.out.print(entry.getKey() + " : ");
            for (int val : entry.getValue()) {
                System.out.print(val + " ");
            }
            System.out.println();
        }
    }

    static Node buildTree(String str) {
        // Example input: "10 N 2 8 9 7 6 3"

        String[] values = str.split(" ");
        Node root = new Node(Integer.parseInt(values[0]));

        Queue<Node> queue = new LinkedList<>();
        queue.add(root);

        int i = 1;

        while (!queue.isEmpty() && i < values.length) {
            Node current = queue.poll();

            if (!values[i].equals("N")) {
                current.left = new Node(Integer.parseInt(valu
es[i]));
                queue.add(current.left);
            }
            i++;

            if (i < values.length && !values[i].equals("N"))
{
                current.right = new Node(Integer.parseInt(val
ues[i]));
                queue.add(current.right);
            }
            i++;
```

```
        }

        return root;
    }

    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
            this.left = null;
            this.right = null;
        }
    }
}
```

Print Top view from vertical order Traversal :

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Node root = buildTree(sc.nextLine());
        verticalOrderTraversal(root);
    }

    static void verticalOrderTraversal(Node root) {
        if (root == null) return;

        Queue<Node> nodeQueue = new LinkedList<>();
        Queue<Integer> hdQueue = new LinkedList<>();
```

```
        nodeQueue.add(root);
        hdQueue.add(0);

        Map<Integer, List<Integer>> map = new TreeMap<>();

        while (!nodeQueue.isEmpty()) {
            Node node = nodeQueue.poll();
            int hd = hdQueue.poll();

            map.putIfAbsent(hd, new ArrayList<>());
            map.get(hd).add(node.data);

            if (node.left != null) {
                nodeQueue.add(node.left);
                hdQueue.add(hd - 1);
            }

            if (node.right != null) {
                nodeQueue.add(node.right);
                hdQueue.add(hd + 1);
            }
        }

        System.out.println("Top View : ");

        for (Map.Entry<Integer, List<Integer>> entry : map.en
trySet()) {
            System.out.print(entry.getValue().get(0) + " ");
        }
    }

    static Node buildTree(String str) {
        // Example: "10 N 2 8 9 7 6 3"

        String[] values = str.split(" ");
        Node root = new Node(Integer.parseInt(values[0]));
```

```
        Queue<Node> queue = new LinkedList<>();
        queue.add(root);

        int i = 1;

        while (!queue.isEmpty() && i < values.length) {
            Node current = queue.poll();

            if (!values[i].equals("N")) {
                current.left = new Node(Integer.parseInt(valu
es[i]));
                queue.add(current.left);
            }
            i++;

            if (i < values.length && !values[i].equals("N"))
{
                current.right = new Node(Integer.parseInt(val
ues[i]));
                queue.add(current.right);
            }
            i++;
        }

        return root;
    }

    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
            this.left = null;
            this.right = null;
```

```
        }
    }
}
```

Recover BST , binary tree level order data insertion , recover bst logic, print level wise elements:

TC : O(n)
SC :

Average case
O(log n)
Worst case
O(n)

```
import java.util.*;

public class Main {

    static Node first, middle, last, prev;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        Node root = buildTree(str);

        recoverBst(root);
        levelOrderTraversal(root);
    }

    static void levelOrderTraversal(Node root) {
        Queue<Node> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
```

```
            Node curr = queue.poll();
            if (curr != null) {
                System.out.print(curr.data + " ");
                queue.add(curr.left);
                queue.add(curr.right);
            }
        }
    }

    static Node buildTree(String str) {
        String[] strip = str.split(" ");
        Queue<Node> queue = new LinkedList<>();

        Node root = new Node(Integer.parseInt(strip[0]));
        queue.add(root);

        int i = 1;

        while (!queue.isEmpty() && i < strip.length) {
            Node curr = queue.poll();

            if (!strip[i].equals("N")) {
                curr.left = new Node(Integer.parseInt(strip
[i]));
                queue.add(curr.left);
            }
            i++;

            if (i < strip.length && !strip[i].equals("N")) {
                curr.right = new Node(Integer.parseInt(strip
[i]));
                queue.add(curr.right);
            }
            i++;
        }
```

```
        return root;
    }

    static void recoverBst(Node root) {
        first = middle = last = prev = null;
        detect(root);

        if (first != null && last != null) {
            int temp = first.data;
            first.data = last.data;
            last.data = temp;
        } else if (first != null && middle != null) {
            int temp = first.data;
            first.data = middle.data;
            middle.data = temp;
        }
    }

    static void detect(Node root) {
        if (root == null) return;

        detect(root.left);

        if (prev != null && prev.data > root.data) {
            if (first == null) {
                first = prev;
                middle = root;
            } else {
                last = root;
            }
        }

        prev = root;
        detect(root.right);
    }
```

```
    static class Node {
        int data;
        Node left, right;

        Node(int d) {
            data = d;
        }
    }
}
```

Boundary Level Traversal Anti-clock wise:

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        Node root = buildTree(str);

        System.out.println("Inorder Traversal:");
        inOrder(root);

        System.out.println("\n\nLevel Order Traversal:");
        levelOrderTraversal(root);

        System.out.println("\n\nBoundary Traversal:");
        boundaryTraversal(root);
    }

    static Node buildTree(String str) {
        String[] arr = str.split(" ");
        Queue<Node> queue = new LinkedList<>();
```

```
        Node root = new Node(Integer.parseInt(arr[0]));
        queue.add(root);

        int i = 1;

        while (!queue.isEmpty() && i < arr.length) {
            Node curr = queue.poll();

            if (!arr[i].equals("N")) {
                curr.left = new Node(Integer.parseInt(arr
[i]));
                queue.add(curr.left);
            }
            i++;

            if (i < arr.length && !arr[i].equals("N")) {
                curr.right = new Node(Integer.parseInt(arr
[i]));
                queue.add(curr.right);
            }
            i++;
        }

        return root;
    }

    static void inOrder(Node root) {
        if (root == null) return;
        inOrder(root.left);
        System.out.print(root.data + " ");
        inOrder(root.right);
    }

    static void levelOrderTraversal(Node root) {
        Queue<Node> queue = new LinkedList<>();
```

```
        queue.add(root);

        while (!queue.isEmpty()) {
            Node curr = queue.poll();
            if (curr != null) {
                System.out.print(curr.data + " ");
                queue.add(curr.left);
                queue.add(curr.right);
            }
        }
    }

    static void boundaryTraversal(Node root) {
        if (root == null) return;

        System.out.print(root.data + " ");

        printLeft(root.left);
        printLeaves(root);
        printRight(root.right);
    }

    static void printLeaves(Node root) {
        if (root == null) return;

        printLeaves(root.left);

        if (root.left == null && root.right == null) {
            System.out.print(root.data + " ");
        }

        printLeaves(root.right);
    }

    static void printLeft(Node root) {
        if (root == null) return;
```

```
        if (root.left != null || root.right != null) {
            System.out.print(root.data + " ");
        }

        if (root.left != null) {
            printLeft(root.left);
        } else {
            printLeft(root.right);
        }
    }

    static void printRight(Node root) {
        if (root == null) return;

        if (root.right != null) {
            printRight(root.right);
        } else {
            printRight(root.left);
        }

        if (root.left != null || root.right != null) {
            System.out.print(root.data + " ");
        }
    }

    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
    }
}
```

Boundary Level Traversal Clock wise:

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        Node root = buildTree(str);

        System.out.println("Boundary Traversal:");
        boundaryTraversal(root);
    }

    static Node buildTree(String str) {
        String[] arr = str.split(" ");
        Queue<Node> queue = new LinkedList<>();

        Node root = new Node(Integer.parseInt(arr[0]));
        queue.add(root);

        int i = 1;

        while (!queue.isEmpty() && i < arr.length) {
            Node curr = queue.poll();

            if (!arr[i].equals("N")) {
                curr.left = new Node(Integer.parseInt(arr
[i]));
                queue.add(curr.left);
            }
            i++;
```

```
            if (i < arr.length && !arr[i].equals("N")) {
                curr.right = new Node(Integer.parseInt(arr
[i]));
                queue.add(curr.right);
            }
            i++;
        }

        return root;
    }

    static void boundaryTraversal(Node root) {
        if (root == null) return;

        System.out.print(root.data + " ");

        printRight(root.right);
        printLeaves(root);
        printLeft(root.left);
    }

    static void printLeaves(Node root) {
        if (root == null) return;

        printLeaves(root.right);

        if (root.left == null && root.right == null) {
            System.out.print(root.data + " ");
        }

        printLeaves(root.left);
    }

    static void printLeft(Node root) {
        if (root == null) return;
```

```
        if (root.left != null) {
            printLeft(root.left);
        } else {
            printLeft(root.right);
        }

        if (root.left != null || root.right != null) {
            System.out.print(root.data + " ");
        }
    }

    static void printRight(Node root) {
        if (root == null) return;

        if (root.left != null || root.right != null) {
            System.out.print(root.data + " ");
        }

        if (root.right != null) {
            printRight(root.right);
        } else {
            printRight(root.left);
        }
    }

    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
    }
}
```

Left view and Right View of Binary Tree :

Time Complexity

O(n)

Space Complexity

O(n)

Note : questions might ask

Left view without root or leaf node.

Right view without root or leaf node .

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String str = sc.nextLine();

        Node root = buildTree(str);

        System.out.println("Inorder:");
        inOrder(root);

        System.out.println("\n\nLevel Order:");
        levelOrderTraversal(root);

        System.out.println("\n\nNodes Without Siblings:");
        printNWS(root);

        List<Integer> rightView = new ArrayList<>();
        findRightView(root, 0, rightView);

        System.out.println("\nRight View:");
        for (int x : rightView) System.out.print(x + " ");
```

```
        List<Integer> leftView = new ArrayList<>();
        findLeftView(root, 0, leftView);

        System.out.println("\nLeft View:");
        for (int x : leftView) System.out.print(x + " ");
    }

    static Node buildTree(String str) {
        String[] arr = str.split(" ");
        Queue<Node> queue = new LinkedList<>();

        Node root = new Node(Integer.parseInt(arr[0]));
        queue.add(root);

        int i = 1;

        while (!queue.isEmpty() && i < arr.length) {
            Node curr = queue.poll();

            if (!arr[i].equals("N")) {
                curr.left = new Node(Integer.parseInt(arr
[i]));
                queue.add(curr.left);
            }
            i++;

            if (i < arr.length && !arr[i].equals("N")) {
                curr.right = new Node(Integer.parseInt(arr
[i]));
                queue.add(curr.right);
            }
            i++;
        }

        return root;
```

```
    }

    static void inOrder(Node root) {
        if (root == null) return;
        inOrder(root.left);
        System.out.print(root.data + " ");
        inOrder(root.right);
    }

    static void levelOrderTraversal(Node root) {
        Queue<Node> queue = new LinkedList<>();
        queue.add(root);

        while (!queue.isEmpty()) {
            Node curr = queue.poll();
            if (curr != null) {
                System.out.print(curr.data + " ");
                queue.add(curr.left);
                queue.add(curr.right);
            }
        }
    }

    static void printNWS(Node root) {
        if (root == null) return;

        if (root.left != null && root.right != null) {
            printNWS(root.left);
            printNWS(root.right);
        } else if (root.left != null) {
            System.out.print(root.left.data + " ");
            printNWS(root.left);
        } else if (root.right != null) {
            System.out.print(root.right.data + " ");
            printNWS(root.right);
        }
```

```
    }

    static void findRightView(Node root, int depth, List<Inte
ger> res) {
        if (root == null) return;

        if (depth == res.size()) res.add(root.data);

        findRightView(root.right, depth + 1, res);
        findRightView(root.left, depth + 1, res);
    }

    static void findLeftView(Node root, int depth, List<Integ
er> res) {
        if (root == null) return;

        if (depth == res.size()) res.add(root.data);

        findLeftView(root.left, depth + 1, res);
        findLeftView(root.right, depth + 1, res);
    }

    static class Node {
        int data;
        Node left, right;

        Node(int data) {
            this.data = data;
        }
    }
}
```

Graphs

Graph BFS and DFS :

Time Complexity
O(V + E)
Space Complexity
O(V + E)

```
import java.util.*;

public class Main {

    static int nVertices, nEdges;
    static List<List<Integer>> graph;
    static boolean[] dfsVisited;
    static List<Integer> dfsResult = new ArrayList<>();

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        nVertices = sc.nextInt() + 1;
        nEdges = sc.nextInt();

        dfsVisited = new boolean[nVertices];
        graph = new ArrayList<>();

        for (int i = 0; i < nVertices; i++)
            graph.add(new ArrayList<>());

        for (int i = 0; i < nEdges; i++) {
            int src = sc.nextInt();
            int dest = sc.nextInt();
            graph.get(src).add(dest);
            graph.get(dest).add(src); // Undirected
        }

        int startVertex = sc.nextInt();
```

```
        System.out.println("BFS:");
        bfs(startVertex);

        System.out.println("\nDFS:");
        dfs(startVertex);
        System.out.println(dfsResult);
    }

    static void dfs(int v) {
        if (dfsVisited[v]) return;

        dfsVisited[v] = true;
        dfsResult.add(v);

        for (int nbr : graph.get(v))
            dfs(nbr);
    }

    static void bfs(int start) {
        Queue<Integer> q = new LinkedList<>();
        boolean[] visited = new boolean[nVertices];
        List<Integer> result = new ArrayList<>();

        q.add(start);
        visited[start] = true;

        while (!q.isEmpty()) {
            int curr = q.poll();
            result.add(curr);

            for (int nbr : graph.get(curr)) {
                if (!visited[nbr]) {
                    q.add(nbr);
                    visited[nbr] = true;
                }
```

```
            }
        }

        System.out.println(result);
    }
}
```

Using Explict stack DFS. Explore last connected neighbour :

```
import java.util.*;

public class Main {

    static int nVertices, nEdges;
    static List<List<Integer>> graph;
    static boolean[] visited;
    static List<Integer> dfsResult = new ArrayList<>();

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        nVertices = sc.nextInt() + 1;
        nEdges = sc.nextInt();

        graph = new ArrayList<>();
        visited = new boolean[nVertices];

        for (int i = 0; i < nVertices; i++)
            graph.add(new ArrayList<>());

        for (int i = 0; i < nEdges; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
```

```
            graph.get(u).add(v);
            graph.get(v).add(u);
        }

        int start = sc.nextInt();

        System.out.println("DFS using Stack:");
        dfsStack(start);

        System.out.println("\nDFS using Recursion:");
        dfs(start);
        System.out.println(dfsResult);
    }

    static void dfs(int v) {
        if (visited[v]) return;

        visited[v] = true;
        dfsResult.add(v);

        for (int nbr : graph.get(v))
            dfs(nbr);
    }

    static void dfsStack(int start) {
        Stack<Integer> st = new Stack<>();
        boolean[] visited = new boolean[nVertices];
        List<Integer> result = new ArrayList<>();

        st.push(start);

        while (!st.isEmpty()) {
            int curr = st.pop();

            if (visited[curr]) continue;
```

```
            visited[curr] = true;
            result.add(curr);

            for (int nbr : graph.get(curr))
                st.push(nbr);
        }

        System.out.println(result);
    }
}
```

Dial’s Algorithm : (Single source shortest path Algo):

Time Complexity
O(E × W)
Space Complexity
O(V + E + W×E)

```
import java.util.*;

public class Main {

    static int nVertices, nEdges, maxLatency, bucketSize;
    static List<List<Edge>> graph;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        nVertices = sc.nextInt();
        nEdges = sc.nextInt();
        maxLatency = sc.nextInt();

        bucketSize = maxLatency * nEdges + 1;

        graph = new ArrayList<>();
```

```
        for (int i = 0; i < nVertices; i++)
            graph.add(new ArrayList<>());

        for (int i = 0; i < nEdges; i++) {
            int u = sc.nextInt();
            int v = sc.nextInt();
            int w = sc.nextInt();

            graph.get(u).add(new Edge(v, w));
            graph.get(v).add(new Edge(u, w));
        }

        int src = sc.nextInt();
        dialsAlgo(src);
    }

    static void dialsAlgo(int src) {
        List<Integer>[] bucket = new ArrayList[bucketSize];

        for (int i = 0; i < bucketSize; i++)
            bucket[i] = new ArrayList<>();

        int[] dist = new int[nVertices];
        Arrays.fill(dist, Integer.MAX_VALUE);

        dist[src] = 0;
        bucket[0].add(src);

        int idx = 0;

        while (idx < bucketSize) {
            while (idx < bucketSize && bucket[idx].isEmpty())
                idx++;

            if (idx == bucketSize) break;
```

```
            int u = bucket[idx].remove(0);

            if (idx > dist[u]) continue;

            for (Edge e : graph.get(u)) {
                int v = e.dest;
                int w = e.weight;

                if (dist[v] > dist[u] + w) {
                    dist[v] = dist[u] + w;
                    bucket[dist[v]].add(v);
                }
            }
        }

        for (int i = 0; i < nVertices; i++) {
            if (dist[i] == Integer.MAX_VALUE)
                System.out.println("to " + i + " INF");
            else
                System.out.println("to " + i + " " + dist
[i]);
        }
    }

    static class Edge {
        int dest, weight;

        Edge(int d, int w) {
            dest = d;
            weight = w;
        }
    }
}
```

Bellman Ford Algo :

Time Complexity

O(V × E)

Space Complexity

O(V + E)

```
import java.util.*;

public class Main {

    static int nVertices, nEdges;
    static List<List<Edge>> graph;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        nVertices = sc.nextInt();
        nEdges = sc.nextInt();

        graph = new ArrayList<>();

        for (int i = 0; i < nVertices; i++)
            graph.add(new ArrayList<>());

        for (int i = 0; i < nEdges; i++) {
            int src = sc.nextInt();
            int dest = sc.nextInt();
            int weight = sc.nextInt();

            graph.get(src).add(new Edge(dest, weight));
        }

        int source = sc.nextInt();
        shortestPath(source);
```

```
    }

    static void shortestPath(int source) {
        int[] dist = new int[nVertices];
        Arrays.fill(dist, Integer.MAX_VALUE);

        dist[source] = 0;

        // Relax edges V-1 times
        for (int i = 0; i < nVertices - 1; i++) {
            for (int u = 0; u < nVertices; u++) {
                for (Edge e : graph.get(u)) {
                    int v = e.dest;
                    int w = e.weight;

                    if (dist[u] != Integer.MAX_VALUE && dist
[v] > dist[u] + w) {
                        dist[v] = dist[u] + w;
                    }
                }
            }
        }

        // Check for negative cycle
        for (int u = 0; u < nVertices; u++) {
            for (Edge e : graph.get(u)) {
                int v = e.dest;
                int w = e.weight;

                if (dist[u] != Integer.MAX_VALUE && dist[v] >
dist[u] + w) {
                    System.out.println("Negative Cycle Exist
s");
                    return;
                }
            }
```

```
        }

        // Print result
        for (int i = 0; i < nVertices; i++) {
            if (dist[i] == Integer.MAX_VALUE)
                System.out.println("To " + i + " INF");
            else
                System.out.println("To " + i + " " + dist
[i]);
        }
    }

    static class Edge {
        int dest, weight;

        Edge(int dest, int weight) {
            this.dest = dest;
            this.weight = weight;
        }
    }
}
```

Topological sort using DFS and Kahn Algo :

Time Complexity
O(V + E)
Space Complexity
O(V + E)

```
import java.util.*;

public class Main {

    static int nVertices, nEdges;
    static List<List<Integer>> graph;
```

```
    static boolean[] visited;
    static Stack<Integer> st = new Stack<>();

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        nVertices = sc.nextInt();
        nEdges = sc.nextInt();

        graph = new ArrayList<>();
        visited = new boolean[nVertices];

        for (int i = 0; i < nVertices; i++)
            graph.add(new ArrayList<>());

        for (int i = 0; i < nEdges; i++)
            graph.get(sc.nextInt()).add(sc.nextInt());

        for (int i = 0; i < nVertices; i++)
            if (!visited[i])
                dfsTopo(i);

        System.out.println("DFS Topological Sort:");
        while (!st.isEmpty())
            System.out.print(st.pop() + " ");

        System.out.println("\nKahn’s Algorithm:");
        kahnAlgo();
    }

    static void dfsTopo(int v) {
        if (visited[v]) return;

        visited[v] = true;

        for (int nbr : graph.get(v))
```

```
            dfsTopo(nbr);

        st.push(v);
    }

    static void kahnAlgo() {
        int[] indegree = new int[nVertices];

        for (int i = 0; i < nVertices; i++)
            for (int nbr : graph.get(i))
                indegree[nbr]++;

        Queue<Integer> q = new LinkedList<>();

        for (int i = 0; i < nVertices; i++)
            if (indegree[i] == 0)
                q.add(i);

        List<Integer> res = new ArrayList<>();

        while (!q.isEmpty()) {
            int curr = q.poll();
            res.add(curr);

            for (int nbr : graph.get(curr)) {
                indegree[nbr]--;
                if (indegree[nbr] == 0)
                    q.add(nbr);
            }
        }

        if (res.size() != nVertices)
            System.out.println("Cycle exists");
        else
            System.out.println(res);
```

```
    }
}
```

Binary Heap insert, Delete :

Time Complexity
O(n log n)
Space Complexity
O(1)

```
import java.util.*;

public class Main {

    static int[] heap = new int[100];
    static int size = 0;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        for (int i = 0; i < n; i++)
            insert(sc.nextInt());

        printHeap();

        System.out.println("Deleted: " + delete());

        printHeap();
    }

    static void insert(int val) {
        heap[size] = val;
        int i = size;
```

```
        size++;

        while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
            swap(i, (i - 1) / 2);
            i = (i - 1) / 2;
        }
    }

    static int delete() {
        if (size == 0) return -1;

        int root = heap[0];
        heap[0] = heap[size - 1];
        size--;

        heapify(0);
        return root;
    }

    static void heapify(int i) {
        int largest = i;
        int l = 2 * i + 1, r = 2 * i + 2;

        if (l < size && heap[l] > heap[largest]) largest = l;
        if (r < size && heap[r] > heap[largest]) largest = r;

        if (largest != i) {
            swap(i, largest);
            heapify(largest);
        }
    }

    static void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
```

```
    }

    static void printHeap() {
        for (int i = 0; i < size; i++)
            System.out.print(heap[i] + " ");
        System.out.println();
    }
}
```

Binary Heap sort MAX HEAP ASC ORDER :

```
import java.util.*;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];

        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        // Build heap
        for (int i = n / 2 - 1; i >= 0; i--)
            heapify(arr, n, i);

        // Sort
        for (int i = n - 1; i > 0; i--) {
            swap(arr, 0, i);
            heapify(arr, i, 0);
        }

        for (int x : arr)
```

```
            System.out.print(x + " ");
    }

    static void heapify(int[] arr, int n, int i) {
        int largest = i;
        int l = 2 * i + 1, r = 2 * i + 2;

        if (l < n && arr[l] > arr[largest]) largest = l;
        if (r < n && arr[r] > arr[largest]) largest = r;

        if (largest != i) {
            swap(arr, i, largest);
            heapify(arr, n, largest);
        }
    }

    static void swap(int[] arr, int i, int j) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```